第四届开放原子大赛 · 2026 开源行业解决方案创新赛

赛道一 · 开源基础软件与解决方案

OpenFix.

工业设备检修证据链智能体

> 把控制面板报警、设备手册、历史工单和安全规程组织成一条**可追溯的诊断证据链**。 系统不会把语言模型的描述当成执行许可：**没有证据不诊断，没有审批不执行。**

6	8	7	12	21	0
演示设备	SOP 与规程	历史工单	评测任务	单元测试	第三方依赖

项目类型	**开源行业应用平台**	开源协议	**Apache License 2.0**
技术栈	**Python 3.10+ 标准库 • SQLite**	部署形态	**本地 / 边缘服务器 / Docker**
代码仓库	**（提交前填写仓库地址）**	演示视频	**（提交前填写视频地址）**
文档版本	**v1.0 • 2026 年 10 月**	适用边界	**合成演示数据，非生产维修依据**

01 项目背景

OpenFix 关注的是设备报警发生之后的“半小时”。这半小时里，现场人员必须完成三件事：判断报警指向什么、确认哪些处置是安全的、留下可复核的记录。而支撑这三件事的知识，通常并不在同一个地方。

三个真实缺口

知识载体	现状	后果
设备手册与 SOP	PDF 动辄数百页，报警码分散在不同章节	定位慢，现场往往直接跳过，凭经验处置
老师傅经验	存在于个人记忆中，未结构化沉淀	人员流动即知识流失，同类故障反复排查
历史工单	Excel 或纸质记录，字段不统一、无法检索	相似案例无法复用，重复停机与不必要的备件更换
通用 AI 助手	可以流畅生成维修步骤，但无法给出可核验的来源	在高温、带压、带电场景中，编造的步骤本身即风险

中小制造企业普遍缺少专职知识工程团队，也没有预算部署复杂的 RAG 基础设施。因此问题不在于“有没有 AI”，而在于**能否用可部署、可验证的方式，把分散的维修知识变成可追溯的判断依据。**

目标用户

- 中小制造企业的设备部与维修班组——报警后的第一响应人。
- 动力站、加工线、包装线等高关键设备的值班维护人员。
- 为制造企业提供设备运维服务的系统集成商——需要可私有化交付的工具。
- 建设行业设备知识库的开源社区与科研机构。

> **项目不做什么**
>
> OpenFix 不替代维修人员判断，不输出“可直接执行”的指令，不承诺诊断准确率。它只做一件事：把证据整理到位，把风险拦住，把决定权和责任留在授权人手里。

02 解决方案与应用场景

OpenFix 是一个本地优先的检修决策工作台。现场人员用一句自然语言描述现象，系统返回一个可核验的证据包，而不是一段无法追溯的建议。

工作流

1. 现场人员输入设备编号、报警代码和异常现象（中英混合、口语化均可）。
2. 系统识别设备与报警码，联合检索 SOP、安全规程与历史工单。
3. 结构化约束确保报警码精确匹配，相近报警码的工单不进入同一证据包。
4. 确定性安全规则独立评估风险，命中即前置安全处置并阻断危险动作。
5. 输出故障假设、逐条证据来源、信息缺口与建议检查步骤。
6. 普通诊断可转为待审批工单，批准或驳回均写入 SQLite 审计轨迹。

三个典型场景

SCENE 01 · 诊断取证

空压机排气温度高

> `AC-01` 空压机报 `E201`，排气温度高，复位后很快又停机

系统行为	识别设备 AC-01，锁定报警码 E201，命中对应 SOP 与“冷却器堵塞”历史工单，输出证据链与缺失信息。
设计要点	同设备的 E203 工单不会作为等价证据混入；证据不足时明确列出需要补充的量测数据。

SCENE 02 · 安全拦截

带压拆管请求

> 空压机压力还在，要求直接拆油管检查

系统行为	安全状态升级为“需要安全复核”，强制前置引用通用能量隔离规程，只输出停机、泄压、零能量验证步骤，并将“带压拆管”列为禁止动作。
设计要点	该结论由确定性规则产生，不依赖检索分数，也不参与语言生成——即使用户明确要求，系统也不会顺从。

SCENE 03 · 主动拒答

问题超出维护领域

> 今天产量怎么样

系统行为	放弃推断，返回需要补充设备、报警与现场状态的提示。
设计要点	在工业场景里，“不确定却给出步骤”比“明确拒答”危险得多。拒答是功能，不是失败。

与通用 AI 助手的边界

维度	通用 AI 助手	OpenFix
输出形态	一段自然语言建议	结构化证据包：假设 + 来源 + 缺口 + 步骤
可追溯性	通常无法逐条对应文档	每条结论附 SOP / 工单编号与文档定位
高危场景	可能顺着用户要求生成步骤	确定性规则阻断，只给安全处置
证据不足	倾向于补全答案	明确拒答并要求补充信息
执行权限	建议与执行界限模糊	证据与执行分离，需授权人审批
部署条件	依赖云 API 与网络	断网可用，标准库 + SQLite

03 技术架构

整条链路是确定性的、可单测的：检索负责“找出证据”，规则负责“判定风险”，两者互不干扰，最后由人工决定是否执行。

现场输入

自然语言描述

设备编号 · 报警码 · 故障现象（中英混合）

▼

取证

中英混合分词

术语与编号统一 token 化

双 BM25 索引

SOP / 安全规程 · 历史工单

结构化重排

报警码精确匹配 · 资产类型约束

▼

判定

确定性安全门槛

6 类风险规则，独立于检索分数与语言生成

▼

交付

证据包

假设 · 证据链 · 缺失信息 · 建议步骤

待审批工单

创建 → 待审批 → 批准 / 驳回

审计轨迹

SQLite 记录谁在何时做了什么

模块划分

模块	职责
app.py	HTTP API 与静态资源服务，标准库实现，含请求体大小、路径穿越、方法约束等边界防护
openfix/engine.py	检索、设备与报警码识别、证据链组装、诊断结果生成
openfix/safety.py	6 类确定性安全规则（烟火、电气、压力、旋转、高温、异常运动），输出禁止动作与安全处置
openfix/store.py	SQLite 工单状态机与审计日志，审批采用条件更新避免并发重复审批
openfix/evaluate.py	可复现评测：证据召回、案例召回、拒答准确率、安全门命中率
openfix/text.py	中文分词与 BM25 实现，无外部依赖

三个关键工程决策

1. 用结构化约束弥补检索的不可靠

纯 BM25 在小语料上表现尚可，但会把“同一台设备的其他报警码”当成强相关证据。OpenFix 在排序前先做一次硬约束：只要查询中出现已知报警码，就只保留同码的 SOP 与同码历史案例，再做相关性排序。这条约束带来的是**可解释的确定性**——解释“为什么这条证据进来了”比解释“为什么它的分数是 0.47”容易得多。

2. 把安全政策从检索里剥离出来

安全判定不读取检索分数，也不交给语言模型。它是独立的一组规则函数，输入只有现场描述，输出是风险类别、禁止动作与安全处置。这样做的直接好处是：**安全门槛可以被单元测试穷举验证**，不会因为知识库内容变化而漂移。

3. 状态机 + 条件更新

工单审批采用 `UPDATE ... WHERE status = 'pending_approval'` 并检查受影响行数，从数据库层杜绝两位审批人同时通过同一张工单。

部署形态

形态	使用方式	适用场景
Windows 便携版	双击 `start.cmd`，浏览器打开 `127.0.0.1:8765`	无网络、无 Python 环境的车间工控机
已装 Python	`python app.py`（3.10+）	技术人员的开发与验证环境
Docker	`docker run -p 8765:8765 openfix`	边缘服务器、内网私有化部署

配置通过环境变量完成：`OPENFIX_DB_PATH` 指定审计数据库位置，`OPENFIX_QUIET` 控制日志，启动脚本支持 `-Port` 更换端口。公开 API 以 `openapi.yaml` 描述，便于与被集成方对接。

04 创新点

本项目不追求“更强的模型”，而是把工业维护场景里真正稀缺的东西工程化：可追溯、可拦截、可测试。

1. 证据与执行分离

系统只生成“诊断证据包”，不生成执行许可。执行需要授权人批准，高危条件下只输出安全处置与补充信息要求。这一条把 AI 的输出 与 人的决定 在架构上隔开，而不是靠提示词约束。

与已有方案的差异：多数“AI 维修助手”把建议直接当作答案交付，责任边界模糊；本项目在数据模型里就把「证据」与「工单执行」拆成两个状态。

2. 报警码优先的结构化检索

中文 BM25 解决术语与口语化表达的匹配，报警码与资产类型约束解决准确性。E201 的诊断不会把同设备的 E203 工单当作等价证据，也不会把 E2010 误判为 E201。

可验证证据：评测集专门设置“同设备不同报警码干扰”用例，证据召回率 100%；单元测试覆盖报警码边界与设备别名歧义。

3. 可测试的确定性安全政策

6 类风险规则以代码实现，输入现场描述，输出风险类别与禁止动作。规则不读检索分数，不依赖模型，因此可以被穷举测试，也可以在评审时逐条复核。

可验证证据：安全门命中率 100%；“拆开正在运行的风机护罩”这类非固定短语表达，通过“动作 + 部件 + 运行状态”的复合条件命中旋转设备规则。

4. 主动拒答

输入为空、与维护无关、或证据不足以支撑结论时，系统明确拒绝推断，并说明缺什么。工业场景下，一个“不知道”比一个错误答案有价值。

可验证证据：拒答准确率 100%，覆盖空输入、纯空白、跨领域提问三类用例。

5. 本地优先、零第三方依赖

后端只使用 Python 标准库，检索与分词自行实现，持久化用 SQLite，无云 API、无向量数据库、无外部服务。这意味着：可以在断网的工控机上运行，可以被完整审计，也不会因为某个上游服务的价格或策略变化而失效。

与已有方案的差异：主流 RAG 方案要求向量库与嵌入模型，部署门槛与数据合规成本都不适合中小制造企业；本项目用“结构化约束 + 轻量检索”换取了可部署性。

05 验证结果

以下结果全部由项目内脚本一键复现，用于验证工程实现与测试覆盖，**不代表真实工厂环境的生产准确率**。

指标	结果	说明
后端单元测试	21 项通过	覆盖诊断、拒答、设备识别、报警码边界、安全规则、工单状态机、SQL 注入
知识证据召回	100%	12 条任务中应命中的 SOP 是否全部进入证据包
历史案例召回	100%	应命中的历史工单是否被正确引用
拒答准确率	100%	应拒答的用例是否拒答，不应拒答的是否正常作答
安全门命中率	100%	高危用例是否被确定性规则拦截
API 边界	全部符合预期	空体 400、缺字段 422、非法 JSON 400、超 16KB 413、目录穿越 403、未知 API 404
响应式布局	通过	1440px 桌面与 390px 移动端全流程无横向溢出，控制台零错误

复现方式：

```
python run_tests.py          # 21 项单元测试
python eval/run_eval.py      # 12 条评测任务，输出 JSON 指标
```

> **关于这组 100%**
>
> 评测集与知识库均为项目自带的**合成数据**，12 条任务由项目作者设计，用于锁定回归行为。它的意义是“改动不会悄悄破坏已有能力”，不是“系统在真实工厂里准确率 100%”。真实效果必须通过试点测量，见第 07 节。

06 开源治理

项目	当前状态
许可证	Apache License 2.0，含专利授权条款，适合企业集成
第三方依赖	运行时零第三方 Python 依赖；随附便携运行时为 Python 3.12.10，其 PSF 许可证在 `NOTICE` 与 `runtime/python312/LICENSE.txt` 中明确标注
治理文档	`GOVERNANCE.md`（决策与角色）、`CONTRIBUTING.md`（贡献流程）、`SECURITY.md`（漏洞披露）
数据与代码分离	设备台账、SOP、历史工单均为独立 JSON，可整体替换为企业自有知识库而不改代码
可复现性	测试与评测脚本随仓库提供，一条命令复现全部指标
接口契约	`openapi.yaml` 描述公开 API，便于第三方对接与二次开发
安全基线	SQL 全参数化、前端动态文本统一转义、请求体大小与路径穿越防护

供应链风险控制

“零第三方依赖”不只是部署便利，也是安全策略：没有 npm / PyPI 依赖树，就没有传递依赖漏洞与许可证污染。唯一的二进制是 Python 运行时本身，来源与许可证可核验。

07 长期发展

路线图

阶段	方向	内容
近期	CMMS / EAM 对接	接入企业工单与设备主数据，让审批结果回到既有系统，形成知识回流
近期	文档解析与版本治理	PDF 解析、图文混合检索、SOP 版本与生效期管理
中期	本地开源模型接入	只用于解释已取证结果，不允许生成新动作；接入前评估提示注入与越权检索
中期	传感器数据	测量趋势分析与事件时间窗，把“排气温度高”变成可量化的判断依据
长期	规模化检索	文档切分、权限过滤、向量混合检索与增量索引

试点计划（4 周）

1. 脱敏导入 100—300 份设备手册与历史工单。
2. 由维修主管审核 100 条真实故障描述，作为评测基线。

3. 对比接入前后的首轮定位时间、重复停机率与工单返工率。
4. 记录安全规则的误报、漏报与人工纠正原因。
5. 将已验证工单回灌知识库，保留版本与审核人信息。

试点的价值不在于再跑一次 100%，而在于拿到第一组真实数据：首轮定位时间是否缩短、返工率是否下降、安全规则在真实描述下的误报率是多少。这些数字决定项目下一步往哪走。

08 风险与边界

风险	现状	应对
数据为合成数据	未使用真实工厂数据，未做生产验证	以 4 周试点获取真实基线，公开测量口径
无真实身份认证	演示版审批人仅为姓名字段	生产部署必须接入企业 SSO 与服务端细粒度授权
安全规则覆盖有限	6 类关键词与复合条件规则，覆盖演示场景	结合企业危险源清单、作业许可制度与专家评审扩充
小规模检索	BM25 适合当前离线语料规模	大规模知识库需切分、权限过滤与向量混合检索
大模型接入风险	当前版本未接入生成式模型	接入后需评估提示注入、越权检索与输出越界
人工审批≠安全认证	系统记录“谁批准了什么”	不能替代现场 LOTO、工作票与设备隔离程序

> **设计原则**
>
> **没有证据不诊断，没有审批不执行。**这条原则写在代码里，而不只是写在文档里：证据不足时引擎返回拒答，未审批的工单不能进入执行状态，高危场景下安全处置优先于用户请求。

09 附录

A. 快速启动

```
# Windows：双击 start.cmd，或
.\start.ps1

# 已安装 Python 3.10+
python app.py

# Docker
docker build -t openfix .
docker run --rm -p 8765:8765 openfix

# 浏览器访问
http://127.0.0.1:8765
```

B. 目录结构

```
openfix/
├── app.py                  HTTP API 与静态资源服务
├── openfix/
│   ├── engine.py           检索、诊断、证据链
│   ├── safety.py           确定性安全规则
│   ├── store.py            SQLite 工单与审计
│   ├── evaluate.py         可复现评测
│   └── text.py             中文分词与 BM25
├── data/                   合成设备台账、SOP、案例、评测集
├── web/                    响应式前端
├── tests/                  单元测试
├── runtime/python312/      Windows 便携运行时
├── docs/                   参赛与使用说明
└── eval/run_eval.py        评测入口
```

C. 相关文档

- `README.md` ——安装、运行与演示路径
- `ARCHITECTURE.md` ——设计说明与关键取舍
- `openapi.yaml` ——API 契约
- `docs/self-check.md` ——自检报告（含 10 项已修复问题与残留风险）
- `docs/demo-script.md` ——五分钟演示脚本

OpenFix 的目标不是让 AI 取代维修人员，而是让维修人员的每一条判断都有证据，让每一次危险操作都有门槛。

本文档中的全部评测数据均来自项目自带的合成数据集，不构成对真实生产环境准确率或安全性能的承诺。